1/ Show in hex notation:

a. packed decimal format for 23. → stores each digit in nibble

⇒ 2 → 0x2 , 3 → 0x3 } combined single byte = 0010 0011
0010 0011 2 3

∴ 23 → 0x23.

b) ASCII char for 23:

ASCII code for '2' → 0x32

'3' → 0x33

∴ 32 33 → space

2/ For each of the following packed decimal, show decimal val:

a) 0111 0011 0000 1001

7 3 0 9

b)

c)

3) A MP (μproc) has words of 1 byte (8 bit). The smallest & largest integer rep b

a. Unsigned. 11111111

Range → 0 to 255 ∴ All 8 bits rep the

magnitude → $2^8 = 256$ distinct vals
from 0 to 255.

b. Sign - Magnitude

Range : −127 to +127

MSB : 0 → +ve, 1 → -ve

Remaining 7 : Magnitude.

c. One's comp:

−127 to +127

d. 2's comp:

−128 to +127

MSB → sign.

-ve num rep by inverting all bit & adding one → 2's comp.

e. Unsigned packed decimal.

→ 0 to 99

stores 2 decimal digits / byte

each nibble 4 bits ↓ 1 digit

45 → 0100 0101

f. Signed packed decimal.

→ −0 to +9

1 nibble for sign. 1 for digit.

for +7 → 0111 1100 (hex 0x7C)

↓ ↓

7 +ve

−7 → 0111 1101 (hex 0x7D)

↓

−ve

④ 1608 → 0001 0110 1001 1000

1786 0001 0111 1000 0110

exceed 9 +6

0010 1101 1 0001 1110 →

1 0110 0110 0110

0011 0100 1000 0100

0011 0100 1000 0100

3 4 8 4

⑤ 10s complement → subtract each digit from 9. and add 1 to result.

0736

−0326 ⟶ 9999

0326

9673

+1

9674

0736

9674

(1) 0410

9674

Of 1/10

6 $x = \frac{A+B \times C}{D-E \times F}$

0 addrs	1	2	3
PUSH A PUSH B PUSH C MUL ADD PUSH D " E " F MUL SUB DIV POP X ↳ remove from stack, Store in X	LOAD E MUL F STORE T LOAD D SUB T STORE T LOAD B MUL C ADD A DIV T STORE X	MOV Y, E MUL Y, F MOV T, D SUB T, Y MOV Y, B MUL Y, C ADD Y, A DIV Y, T MOV, X, Y	MUL Y, E, F SUB Y, D, Y MUL T, B, C ADD T, A, T DIV x, ~~Y~~, ~~T~~ T, Y

7 a. Data transfer, X → ACC & ACC → X (auto done)

⇒ Use only SUBS x, JUMP X.
wanna end up with AC = x

initial state a 0 (temp storage) X
(helper reg initialized to 0.)

instr	AC	Effect on M(0)	M(x)
SUBS 0	a − 0 = a	a	x

(stores acc's val to M(0))	AC = AC - M(0)	stores new AC	
SUBS 0	0 (AC = a - a) [AC - M(0)]	0	X
SUBS X (x ← AC - x)	0 - x = -x (if we didn't use M(0), we would get a - x, don't want it)	0	-x
SUBS 0	-x - 0 = -x	-x	-x
SUBS 0	0 AC - M(0) = -x + x	0	-x
SUBS x	x AC - M(x) = 0 - (-x)	0	x

↳ copy & move

AC → register

M(0) → A mem loc that we fix to 0.

SUBS Y → subtracts the val at mem loc Y from AC.

↳ AC = AC - M(Y)

M(Y) = AC.

b Addition: Add loc x contents to Acc

⇒

initial	a	0	y	x
instr.	AC	M(0)	M(y)	M(x)
SUBS 0 (Store AC val to M(0))	a	a	y	x
SUBS y	a - y	a	a - y	x

SUBS y	0	a	0	x
SUBS x	-x	a	0	-x
SUBS 0	-x-a	-x-a	0	-x
SUBS y (sweet)	-x-a	-x-a	-x-a (empty acc, done now)	-x
SUBS 0	0	0	-x-a	-x
SUBS x	x	0	-x-a	x
SUBS 0	x	x	-x-a	x
SUBS 0	0	0	-x-a	x
SUBS y	x+a	0	x+a	x

(c) Conditional Branch

(c) Conditional Branch

Assume a memory cell M(COND) is preloaded with the fall-through address L2, and the branch target is L1 when the accumulator (AC) is negative. Then the conditional branch is implemented as follows:

```
SUBS M(COND)    ; AC ← AC − M(COND), and M(COND) becomes (AC − L2)
JUMP M(COND)    ; PC ← M(COND)
```

Explanation:

If AC is negative, subtracting L2 (the default address) from

AC yields L1 (the desired branch target) in two's complement arithmetic; if AC is nonnegative, the subtraction leaves M(COND) equal to L2. The JUMP then transfers control to the address now stored in M(COND).

(d) Logical OR

Using the Boolean identity

A OR B = NOT[(NOT A) AND (NOT B)],

and assuming M(−1) contains −1 (all ones), along with helper memory cells M(NOTA), M(NOTB), and M(TEMP):

1. Compute NOT A:

```
SUBS M(-1)    ; Load -1 into AC
SUBS M(A)     ; AC becomes (-1 - A) = NOT A; store in M(NOTA)
```

2. Compute NOT B:

```
SUBS M(-1)    ; Load -1 into AC again
SUBS M(B)     ; AC becomes (-1 - B) = NOT B; store in M(NOTB)
```

3. Compute (NOT A) AND (NOT B):

- Process the bits (e.g., bit-by-bit arithmetic) to obtain (NOT A) AND (NOT B); store the result in M(TEMP).

4. Compute A OR B:

```
SUBS M(-1)    ; Load -1 into AC
SUBS M(TEMP)  ; AC becomes (-1 - ( (NOT A) AND (NOT B)
```

)) = A OR B
```

*Explanation:*
By computing the two's complement "NOT" of A and B as −1 − A and −1 − B, and then obtaining their bitwise AND (through an appropriate arithmetic method), subtracting this result from −1 produces A OR B.

---

**(e) I/O Operations**
Assume M(IN) and M(OUT) are the memory-mapped addresses for the input and output devices, respectively.

*Input (reading into AC):*
```
SUBS M(0)   ; Use M(0)=0 to clear or set up AC as needed
SUBS M(0)   ; (Double subtraction with 0 helps zero AC)
SUBS M(IN)   ; AC ← 0 − M(IN) = −(Input)
; Additional SUBS M(0) operations can adjust the sign if required.
```

*Output (writing AC to output):*
```
SUBS M(OUT)   ; AC ← AC − M(OUT); the result is stored in M(OUT) (mapped to the output device)
```

*Explanation:*
For input, the device value is read from M(IN) by subtracting
```

it from zero, yielding −(Input) in AC; further operations may be applied to obtain the desired representation. For output, subtracting M(OUT) from AC causes the current AC value to be written to the output device through the memory-mapped location M(OUT).

This completes the implementations for conditional branching, logical OR, and I/O operations using only the two instructions, SUBS and JUMP.

12.8 Use of NOOP (no operation, no effect on proc state just increments PC.

1. Useful for debugging. To interrupt a prog at a particular point, NOOP replaced with a jump to a debug routine

temporary patch on altered prog → replace instrs. with NOOPS

2. Noop introduces a fixed delay (= 1 instr cycle)
→ useful for timing measurements / creating deliberate time delays

3. Can be inserted to pad code, aligns instrs on

word boundaries or subroutines on page boundaries.

4. IN RISC pipelined structures, NOOPS help manage pipeline hazards by filling delay slots to ensure proper instn. sequencing

Let's break down the four uses of NOOP (No Operation) instructions in a structured and easy-to-understand way:

---

1. Debugging with NOOPs

What it means:
A NOOP is a placeholder instruction that does nothing. During development, programmers can insert NOOPs at strategic points in the code. If a bug is discovered, these NOOPs can later be replaced with a **JUMP** instruction to a debugging routine or patched with corrected code.

Example:
Imagine writing a program like this:

```assembly
MOV A, B   ; Move value from B to A
NOOP       ; Placeholder for debugging
ADD C, D   ; Add C and D
```

If there's an error after `MOV A, B`, you can replace the `NOOP` with:

```assembly
JUMP DEBUG_ROUTINE  ; Redirect to a debug function
```

This avoids rewriting large sections of code and minimizes disruption.

Why use NOOPs?
- They act as safe "gaps" in the code.
- Replacing them doesn’t break existing logic.

---

2. Fixed Delays for Timing
What it means:
Each NOOP takes **one instruction cycle** to execute. By inserting multiple NOOPs, you can create precise delays.

Use cases:
- **Hardware synchronization**: Wait for a sensor or peripheral to respond.
- **Timing measurements**: Calibrate how long other instructions take.

Example:
To create a 5-cycle delay:
```assembly
NOOP
NOOP
NOOP
NOOP
NOOP
```

This ensures the processor waits exactly 5 cycles before proceeding.

Why use NOOPs instead of loops?
- **Predictability**: Loops might have variable execution times (due to branch penalties or interrupts).
- **Simplicity**: No need to manage loop counters.

---

3. Padding for Memory Alignment
What it means:
Processors often fetch instructions in fixed-size blocks (e.g., 4-byte "words"). If code isn't aligned to these boundaries, it can slow down execution. NOOPs pad the code to ensure alignment.

Example:
Suppose instructions are 4 bytes long, and your code ends at 2 bytes:
```assembly
MOV A, B   ; 4 bytes
SUB C, D   ; 4 bytes
; Ends at 8 bytes (aligned)
```
But if it ends at 10 bytes:
```assembly
MOV A, B   ; 4 bytes
SUB C, D   ; 4 bytes
NOOP       ; 2 bytes (padding)
NOOP       ; 2 bytes (padding)
```

Now the next instruction starts at 12 bytes (aligned to a 4-byte boundary).

Why align code?
- **Performance**: Misaligned memory accesses take longer.
- **Hardware requirements**: Some processors crash on misaligned accesses.

---

4. Managing Pipeline Hazards in RISC Architectures
What it means:
RISC (Reduced Instruction Set Computer) pipelines process multiple instructions simultaneously. If one instruction depends on the result of a previous one (e.g., a branch), the pipeline might stall. NOOPs fill these gaps to keep the pipeline flowing.

Example:
After a branch instruction, the pipeline might need a **delay slot** to resolve the branch:
```assembly
BRANCH TARGET   ; Jump to TARGET if condition is met
NOOP            ; Delay slot (do nothing)
ADD A, B        ; This executes before the branch resolves
```

proc immediately doesn't know which path to take cz branch cond^n is resolved only in a later pipeline stage.

If no useful instruction can fill the delay slot, a NOOP is used.

Why use NOOPs here?
- **Prevent errors**: Avoid executing invalid instructions

while waiting.
- **Optimize performance**: Keep the pipeline full instead of stalling.

Key Takeaways
- **NOOPs are versatile**: They solve problems like debugging, timing, alignment, and pipeline efficiency.
- **Even "useless" instructions have purpose**: In constrained systems (like the hypothetical SUBS/JUMP computer), NOOPs can be emulated (e.g., `SUBS 0` to subtract zero) to achieve similar effects.

By understanding these use cases, you'll see how NOOPs contribute to robust, efficient, and debuggable code! 🛠️⏱️🔧

12.0 For 5 bit 2's complement integer
↳ -16(10000) to +15(01111)

Bit pattern	Value	Arithmetic left shift	value	Logical left shift	value
00000	0	00000	0	00000	
00001	1	00010	2	00010	
00010	2	00100	4	00100	
00011	3	00110	6	00110	
00100	4	01000	8	01000	
00101	5	01010	10	01010	
00110	6	01100	12	01100	
00111	7	01110	14	01110	
01000	8	~~10000~~ 00000	overflow	10000	
01001	9		"	10010	
01010	10		"	10100	

01011	11		11	10110	
01100	12		11	11000	
01101	13		11	11010	
01110	14		11	11100	
01111	15		11	11110	
10000	-16		11	00000	
10001	-15		11	000	
10010	-14		11		
10011	-13		11		
10100	-12		11		
10101	-11		11		
10110	-10		11		
10111	-9		11		
11000	-8				
11001	-7				
11010	-6				
11011	-5				
11100	-4				
11101	-3				
11110	-2				
11111	-1				

10.9

Bit pattern	Value	Arithmetic left shift	Value	Logical left shift	Value
00000	0	00000	0	00000	0
00001	1	00010	2	00010	2
00010	2	00100	4	00100	4
00011	3	00110	6	00110	6
00100	4	01000	8	01000	8
00101	5	01010	10	01010	10
00110	6	01100	12	01100	12
00111	7	01110	14	01110	14
01000	8	00000	overflow	10000	overflow
01001	9	00010	overflow	10010	overflow
01010	10	00100	overflow	10100	overflow
01011	11	00110	overflow	10110	overflow
01100	12	01000	overflow	11000	overflow
01101	+13	01010	overflow	11010	overflow
01110	14	01100	overflow	11100	overflow
01111	15	01110	overflow	11110	overflow
10000	–16	10000	overflow	00000	overflow
10001	–15	00010	overflow	00010	overflow

10010	–14	10100	overflow -24	00100	overflow
10011	–13	10110	overflow -22	00110	overflow
10100	–12	11000	overflow -20	01000	overflow
10101	–11	11010	overflow -18	01010	overflow
10110	–10	11100	overflow valid	01100	overflow
10111	–9	11110	overflow	01110	overflow
11000	–8	10000	–16	10000	–16
11001	–7	10010	–14	10010	–14
11010	–6	10100	–12	10100	–12
11011	–5	10110	–10	10110	–10
11100	–4	11000	–8	11000	–8
11101	–3	11010	–6	11010	–6
11110	–2	11100	–4	11100	–4
11111	–1	11110	–2	11110	–2

12.10 arithmetic right shift → round toward $-\infty$

12.11

Suppose a stack is to be used by the processor to manage procedure calls and returns.Can the program counter be eliminated by using the top of the stack as a programcounter?

=> Yes, if the stack is only used to hold the return address.

if it is also used to pass parameters,

↳ will only work if CU removes parameter rather than MI.

↳ CPU would need both a parameter & the PC on top of the stack all the time.

Instruction Fetching: If the stack's top holds the return address (acting as the PC), pushing parameters onto the stack can displace this address, complicating the retrieval of

stack can displace this address, complicating the retrieval of the correct next instruction upon returning from a procedure.

Parameter Removal: In architectures where the control unit (hardware) manages parameter removal after procedure execution, integrating the PC with the stack's top is feasible. However, if parameter removal is handled by executed instructions (software), the CPU would need simultaneous access to both parameters and the return address, necessitating separate storage locations for the PC and parameters.

12.12

12.12 The x86 architecture includes an instruction called Decimal Adjust after Addition (DAA). DAA performs the following sequence of instructions:

```
if ((AL AND 0FH) >9) OR (AF = 1)  then
      AL ← AL + 6;
      AF ← 1;
else
      AF ← 0;
endif;
if (AL > 9FH) OR (CF = 1)  then
      AL ← AL + 60H;
      CF ← 1;
else
      CF ← 0;
endif.
```

"H" indicates hexadecimal. AL is an 8-bit register that holds the result of addition of two unsigned 8-bit integers. AF is a flag set if there is a carry from bit 3 to bit 4 in the result of an addition. CF is a flag set if there is a carry from bit 7 to bit 8. Explain the function performed by the DAA instruction.